Ideation Phase

Brainstorm & Idea Prioritization Template

Date	23 September 2026
Team ID	SWTID-2026-5076
Project Name	Script-Controlled ACL – Restrict Record Access Based on Field Value
Maximum Marks	4 Marks

Brainstorm & Idea Prioritization Template:

This brainstorming exercise follows the supplied template and focuses on generating multiple solutions for enforcing record-level access in ServiceNow using ACLs, user roles, and field-value conditions. Ideas are first generated openly, then grouped, and finally prioritized using practical implementation criteria.

Step-1: Team Gathering, Collaboration and Select the Problem Statement

Project context: The project uses a custom Institution Details table with a Branch field (ECE, EEE, CSE). The reference project requires EEE users to access EEE records while administrators retain full access, with separate ACLs controlling create, write, and delete actions.

Selected Problem Statement:

How can we implement a secure and maintainable Script-Controlled ACL solution that restricts record access according to the Branch field value, while allowing administrators full access and controlling read, create, write, and delete operations through appropriate roles?

Step-2: Brainstorm, Idea Listing and Grouping

Following the template's volume-first brainstorming approach, the team can consider the following solution ideas before selecting the most suitable implementation.

No.	Idea	Description	Group
1	Script-based READ ACL	Access is allowed only when the user satisfies the required role/condition; admin receives full access.	Record Visibility
2	Branch-based Data Condition	Use a Branch = EEE condition to restrict which records are exposed to the target users.	Record Visibility
3	Role-based CREATE ACL	Use a dedicated role (bb2) to control who can create records.	Operation Control
4	Role-based WRITE ACL	Use a dedicated role (bb3) to control who can edit records.	Operation Control
5	Role-based DELETE ACL	Use a dedicated role (bb4) to control who can delete records.	Operation Control
6	Admin Full-Access Exception	Allow users with the admin role to bypass the branch restriction.	Exception Handling
7	Layered Role Model	Combine bb1, bb2, bb3, and bb4 so permissions can be	Role Design

		granted incrementally.	
8	Impersonation Testing	Test EEE user, non-role user, and admin behavior to validate the ACL rules.	Testing
9	Negative Access Testing	Verify that users without the required roles cannot view, create, edit, or delete protected records.	Testing
10	Centralized ACL Documentation	Document each ACL's operation, role requirement, condition, and expected outcome.	Maintainability

Idea Grouping Summary

Group	Ideas Included
Record Visibility	Script-controlled READ ACL + Branch data condition
Operation Control	Separate CREATE, WRITE, and DELETE ACLs
Role Design	Layered bb1–bb4 role model
Exception Handling	Administrator full-access rule
Testing	Impersonation and negative-access verification
Maintainability	Clear ACL documentation and expected outcomes

Step-3: Idea Prioritization

Prioritize the ideas using implementation relevance, security impact, ease of testing, and maintainability. The scores below are project-planning judgments for this brainstorming exercise.

No.	Idea	Security Impact (1–5)	Implementation Fit (1–5)	Maintainability / Testability (1–5)	Total	Reason
1	Script-based READ ACL + Branch condition	5	5	5	15	Core requirement
2	Admin full-access exception	5	5	5	15	Required exception
3	Separate CREATE/WRITE/DELETE ACLs	5	5	4	14	Operation control
4	Layered bb1–bb4 role model	4	5	4	13	Granular permissions
5	Impersonation testing	4	4	5	13	Validation
6	Negative-access testing	4	5	4	13	Security verification
7	Centralized ACL documentation	3	3	5	11	Maintainability

Prioritized Implementation Plan

1. 1. Configure the READ ACL so access is evaluated against the required role and Branch condition.
2. 2. Preserve administrator full access as an explicit exception.
3. 3. Configure separate CREATE, WRITE, and DELETE ACLs using bb2, bb3, and bb4 respectively.
4. 4. Test the layered permissions using EEE users, users without the required role, and administrators.
5. 5. Record expected results and document each ACL for future maintenance.

Expected Outcome:

The brainstorming and prioritization process produces a structured ACL implementation approach in which field values and user roles are evaluated together, while CRUD operations are separately controlled. This aligns with the supplied project outcome that ACLs protect records from unauthorized viewing, modification, creation, and deletion.